

Absolutely. 🔥 This is actually the **right point** to stop treating these as isolated Next.js APIs and build one big mental model:

**Routing → Rendering → Data Fetching → Caching → Revalidation → RSC Payload
→ Browser/Router Cache**

Once these are connected, things like `getServerSideProps`, `getStaticProps`, SSG, SSR, ISR, Server Components, `loading.tsx`, `error.tsx`, Route Handlers, and Router Cache stop feeling like random Next.js terminology.

Lesson 5.9 — App Router vs Pages Router + Rendering Under the Hood

We'll go **deep**, but in layers so it doesn't become a giant soup. 😊

1. First: What does "routing" actually mean?

Suppose your application has:

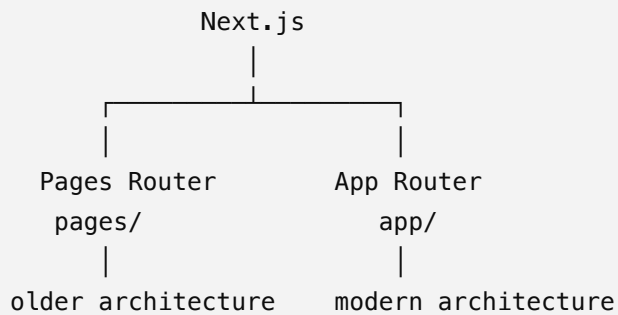
```
/products
/products/123
/products/456
/about
/contact
```



Routing answers:

"When the browser requests this URL, what code should handle it?"

In Next.js, there have historically been **two routing systems**:



Pages Router

```
pages/
├── index.tsx
├── about.tsx
├── products/
│   ├── index.tsx
│   └── [id].tsx
└── api/
    └── products.ts
```

App Router

```
app/
├── page.tsx
├── about/
│   └── page.tsx
├── products/
│   ├── page.tsx
│   └── [id]/
│       └── page.tsx
```

Both can coexist in one Next.js project.

But they represent **two different architectural generations**.

2. Why did Next.js introduce the App Router?

This is the most important historical question.

The Pages Router was designed around:

React page

↓

fetch data

↓

render page

↓

send HTML

↓

hydrate React



It worked extremely well.

But as React evolved, Next.js wanted deeper support for things like:

- React Server Components
- streaming
- nested layouts
- Suspense
- loading UI
- error boundaries
- server-side mutations
- finer-grained caching
- partial client navigation
- keeping layouts mounted while navigating

The Pages Router wasn't designed around these concepts.

So Next.js introduced:

App Router

based on React's newer server/client architecture.

The conceptual shift was:

Pages Router

URL

↓

Page

↓

Render page



versus:

App Router



URL



Route Tree



Layouts + Pages + Loading + Error boundaries



Server Components



RSC Payload + HTML



Client Router Cache

That's a **massive architectural difference**.

3. `pages/*.tsx` vs `app/**/page.tsx`

Let's compare.

Pages Router

`pages/products.tsx`



represents:

`/products`



Example:

```
export default function Products() {  
  return <h1>Products</h1>  
}
```

App Router

`app/products/page.tsx`



represents:

/products



Example:

```
export default function Products() {  
  return <h1>Products</h1>  
}
```

So at the simplest level:

pages/products.tsx

↓

/products

app/products/page.tsx

↓

/products



But what happens around that component is very different.

4. App Router introduces the route tree

Imagine:

```
app/  
├── layout.tsx  
├── page.tsx  
└──  
    ├── products/  
    │   ├── layout.tsx  
    │   ├── page.tsx  
    │   ├── loading.tsx  
    │   └── error.tsx  
    └── [id]/  
        └── page.tsx
```

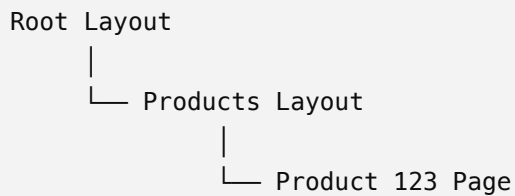


Now imagine requesting:

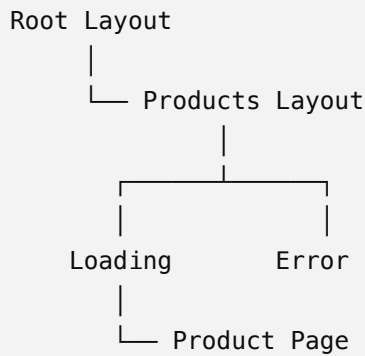
/products/123



Next.js constructs something conceptually like:



And potentially boundaries:



This is why the App Router is called a **nested routing architecture**.

5. Layouts are one of the biggest differences

In the Pages Router, you generally had to build shared layouts yourself.

For example:

Navbar
Sidebar
Page
Footer

With App Router:

app/layout.tsx

can provide the persistent outer structure.

Example:

```
export default function RootLayout({
  children
}: {
  children: React.ReactNode
}) {
```

```
return (  
  <html>  
    <body>  
      <Navbar />  
  
      {children}  
  
      <Footer />  
    </body>  
  </html>  
)  
}
```

Then:

/products



and:

/about



can both live inside that layout.

6. What does "persistent layout" actually mean?

This becomes important for Router Cache.

Suppose:

/products



is currently displayed.

Then user clicks:

/about



The browser doesn't necessarily need to throw away everything and rebuild the entire UI.

Conceptually:

```
Root Layout  
  |  
  └─ Navbar
```





Page

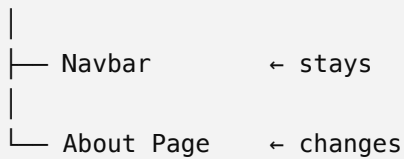
When navigating:

/products
↓
/about



the root layout can remain:

Root Layout



Navbar ← stays
About Page ← changes



This is one reason App Router navigation can feel much more application-like.

7. Now let's get into rendering

This is where I want you to be very precise.

People often say:

"Next.js supports SSR, SSG and ISR."

That's true historically, but it's incomplete for modern App Router.

We need to separate **three concepts**:

A. Where code executes

Server
Client



B. When/how HTML is produced

Static
Dynamic
Streaming



C. Where results are cached

Data Cache
Full Route Cache
Router Cache



These are related, but **they are not the same thing**.

8. Server Components vs Client Components

In App Router:

```
export default function Page() {  
  return <h1>Hello</h1>  
}
```

is a **Server Component by default**.

That means the component executes on the server.

You can explicitly create a Client Component:

```
"use client"  
  
export default function Counter() {  
  return ...  
}
```

Now it participates in client-side React behavior.

9. Server Component ≠ SSR

This distinction is HUGE.

A Server Component means:

| This component's React rendering logic executes on the server.

SSR means:

| HTML is dynamically generated for a request.

These aren't synonyms.

For example, a Server Component can participate in a **statically rendered** route.

So:

```
Server Component
  ↓
can be statically rendered
  OR
can be dynamically rendered
```



That's why don't memorize:

```
Server Component = SSR
```



✗ Wrong.

10. Then what is SSG?

SSG = **Static Site Generation**.

Conceptually:

```
Build time
  ↓
Execute page
  ↓
Generate HTML
  ↓
Store generated result
  ↓
Users receive it later
```



Imagine:

```
/about
```



The page doesn't change.

Next.js can generate it ahead of time.

```
BUILD

/about
  ↓
HTML
```



+
RSC payload

Then request:

Browser → /about



can be served from the cached/generated route.

11. SSR

SSR = **Server-Side Rendering**.

The important difference:

REQUEST
↓
execute/render
↓
HTML
↓
response



Example:

User A → /dashboard
↓
server renders
↓
HTML



Then:

User B → /dashboard
↓
server renders again



assuming the route is dynamic and not otherwise cached.

12. ISR

ISR = **Incremental Static Regeneration**.

Think:

Static generation + regeneration over time.

Suppose:

`/products`

is generated.

You configure a revalidation period.

Conceptually:

```
graph TD
    A[Generate page] --> B[serve cached page]
    B --> C[revalidation period expires]
    C --> D[regenerate]
    D --> E[new version gets cached]
```

So ISR sits conceptually between:

```
graph TD
    SSG[SSG] --- static[static]
    SSR[SSR] --- dynamic[dynamic]
    static --- ISR[ISR]
    dynamic --- ISR
```

13. But here's the modern App Router twist

In modern Next.js, you shouldn't think:

SSG
SSR
ISR

as three completely separate rendering engines.

Instead think:

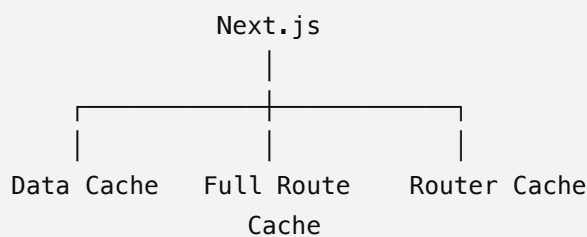
```
graph TD
    A[Route] --> B[How is its data obtained?]
```

↓
Is it static or dynamic?
↓
What gets cached?
↓
When is it revalidated?

That's much closer to the underlying architecture.

14. The three caches

This connects directly to what we learned.



Data Cache

Caches fetched data.

fetch(...)
↓
Data Cache

Full Route Cache

Caches the rendered server output for a route.

Conceptually:

Route
↓
HTML
+
RSC Payload

Router Cache

Client-side cache.

It stores RSC payload segments used by the client router.

15. Now connect rendering to caching

Suppose:

```
/products
```



uses:

```
const products = await fetch("/api/products")
```

Imagine that data is cacheable.

Then:

```
Request
  ↓
Server Component
  ↓
fetch()
  ↓
Data Cache
  ↓
data
  ↓
render React tree
  ↓
RSC Payload + HTML
  ↓
Full Route Cache
```



And after navigation, the browser can hold relevant RSC data in:

```
Router Cache
```



Now we've connected the pieces.

16. The complete first-request flow

Let's say user directly enters:

```
https://example.com/products
```

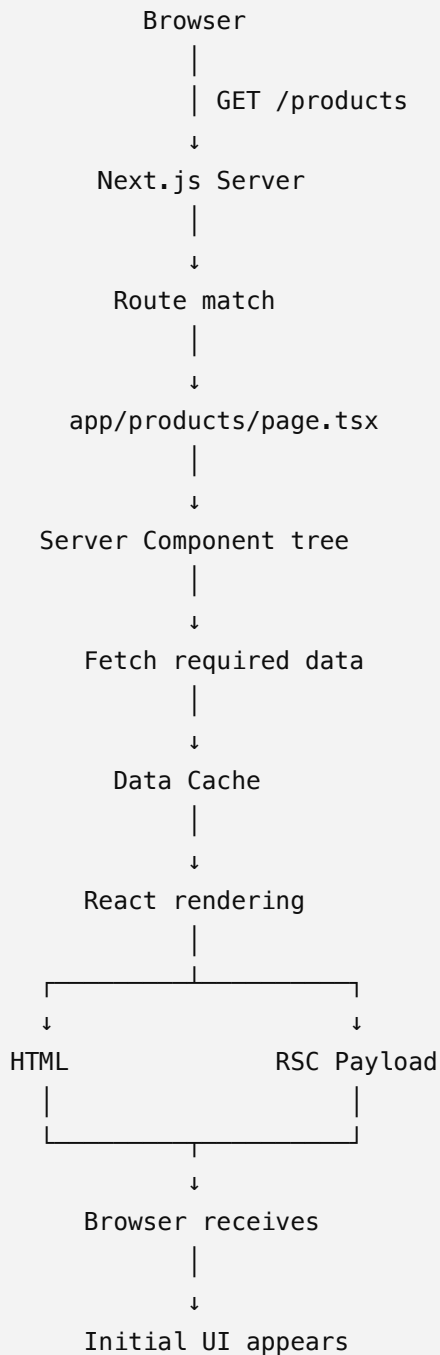


The browser sends:

GET /products



Then conceptually:



And relevant server-rendered route output may be cached in:

Full Route Cache



while the client router maintains its own:

Router Cache



17. What exactly is RSC Payload?

This was one of your earlier sticking points, so let's nail it.

RSC = **React Server Components**.

The RSC Payload is not simply:

"HTML."

It's information React uses on the client to understand the Server Component result/tree and integrate it with Client Components.

Conceptually:

Server Component tree



RSC Payload



Client React



construct/update UI



Initial navigation can involve:

HTML

+

RSC Payload

+

JS for Client Components



Don't think:

RSC Payload = HTML



Think:

HTML = browser's initial document representation

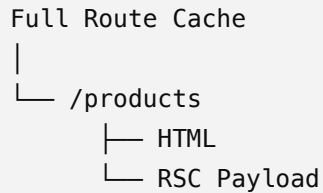


RSC Payload = React's server-component representation/data

18. Why does the Full Route Cache have HTML + RSC Payload?

Because Next.js may cache the rendered route result.

Conceptually:



This allows Next.js to reuse the server-rendered result instead of rendering the route from scratch every time.

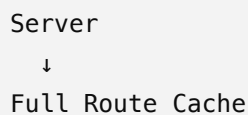
19. Then why does Router Cache also have RSC Payload?

You asked this earlier, and this is the key.

They're at **different locations serving different purposes**.

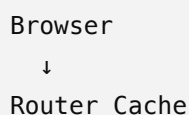
Full Route Cache

Server-side:

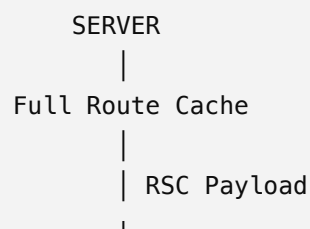


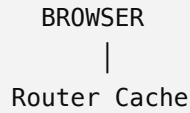
Router Cache

Client-side:



So:





The Router Cache allows the client router to reuse previously received route segments without repeatedly requesting them from the server.

20. Client-side navigation changes everything

Now user is on:

/products



and clicks:

/products/123

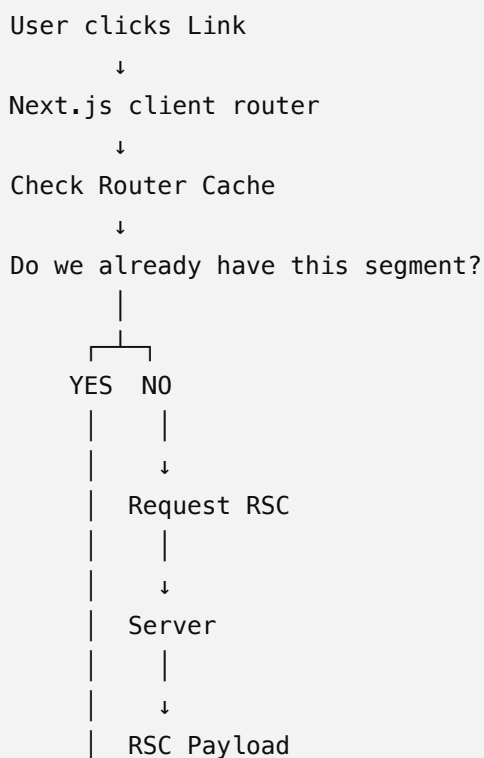


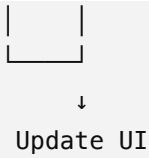
Because you're using:

```
<Link href="/products/123">
```

Next.js can perform client-side navigation.

Conceptually:





This is one of the biggest reasons App Router navigation is powerful.

21. Now: `loading.tsx`

Suppose:

```
app/products/[id]/page.tsx
```



takes time to fetch.

You can have:

```
app/products/[id]/loading.tsx
```



Next.js can show:

```
Loading product...
```



while the server is producing the result.

Conceptually:

```
Request
  ↓
Route
  ↓
Loading boundary
  ↓
"Loading..."
  ↓
Server work continues
  ↓
Product result arrives
  ↓
Replace loading UI
```



This is connected to **Suspense and streaming**.

22. Streaming

Traditional mental model:

```
Server does EVERYTHING
      ↓
HTML complete
      ↓
send everything
```



Streaming:

```
Server starts rendering
      ↓
send available UI
      ↓
send more UI
      ↓
send more UI
```



Imagine:

```
Navbar
Product list loading...
```



Then later:

```
Navbar
Product 1
Product 2
Product 3
```



This is deeply connected to:

```
Suspense
loading.tsx
Server Components
RSC streaming
```



23. error.tsx

Similarly:

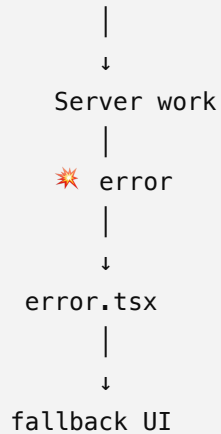
```
app/products/error.tsx
```



creates an error boundary around that route segment.

Conceptually:

Products Route



And because `error.tsx` is a Client Component boundary, you'll commonly see:

```
"use client"
```

at the top.

24. Now let's compare Pages Router data fetching

This is where the famous functions come in.

`getServerSideProps`

Pages Router:

```
export async function getServerSideProps() {
  const data = await fetch(...)

  return {
    props: {
      data
    }
  }
}
```

```
}  
}
```

Conceptually:

```
REQUEST  
  ↓  
getServerSideProps()  
  ↓  
fetch data  
  ↓  
render page  
  ↓  
HTML  
  ↓  
browser
```



That's classic SSR.

25. `getStaticProps`

```
export async function getStaticProps() {  
  const data = await fetch(...)  
  
  return {  
    props: {  
      data  
    }  
  }  
}
```

Conceptually:

```
BUILD  
  ↓  
getStaticProps()  
  ↓  
fetch data  
  ↓  
generate HTML  
  ↓  
store static result
```



That's classic SSG.

26. `getStaticPaths`

This was needed for dynamic routes.

Suppose:

```
pages/products/[id].tsx
```



How does Next.js know which IDs to pre-render?

You tell it:

```
export async function getStaticPaths() {  
  return {  
    paths: [  
      { params: { id: "1" } },  
      { params: { id: "2" } },  
      { params: { id: "3" } }  
    ],  
    fallback: false  
  }  
}
```

Then:

```
Build  
|  
├─ /products/1  
├─ /products/2  
└─ /products/3
```



are generated.

27. `getStaticPaths` + `getStaticProps`

Together:

```
getStaticPaths  
  ↓  
Which pages should exist?  
  ↓  
/products/1  
/products/2  
/products/3  
  ↓
```



```
getStaticProps
```

↓

What data should each page contain?

↓

Generate HTML

So:

```
getStaticPaths = WHICH dynamic pages?
```



```
getStaticProps = WHAT data for each page?
```

Very important distinction.

28. How ISR worked in Pages Router

You could do:

```
return {  
  props: {  
    data  
  },  
  revalidate: 60  
}
```

Meaning conceptually:

```
Generate page  
  ↓  
cache it  
  ↓  
serve cached version  
  ↓  
after 60 sec  
  ↓  
regenerate
```



That's ISR.

29. App Router doesn't use these functions

This is important.

In App Router:

✗ `getServerSideProps`
✗ `getStaticProps`
✗ `getStaticPaths`



Instead, you use things such as:

`fetch()`
`revalidate`
`dynamic`
`generateStaticParams()`



and newer cache/revalidation APIs.

For example:

```
const res = await fetch(  
  "https://api.example.com/products",  
  {  
    next: {  
      revalidate: 60  
    }  
  }  
)
```

Conceptually:

`fetch`
↓
cache data
↓
revalidate after 60 sec



30. `generateStaticParams`

This is the App Router equivalent conceptually related to `getStaticPaths`.

Example:

```
export async function generateStaticParams() {  
  return [  

```

```
{ id: "1" },  
  { id: "2" },  
  { id: "3" }  
]  
}
```

For:

app/products/[id]/page.tsx



this tells Next.js about params that can be generated ahead of time.

So:

Pages Router



getStaticPaths()



dynamic routes to pre-render

versus:

App Router



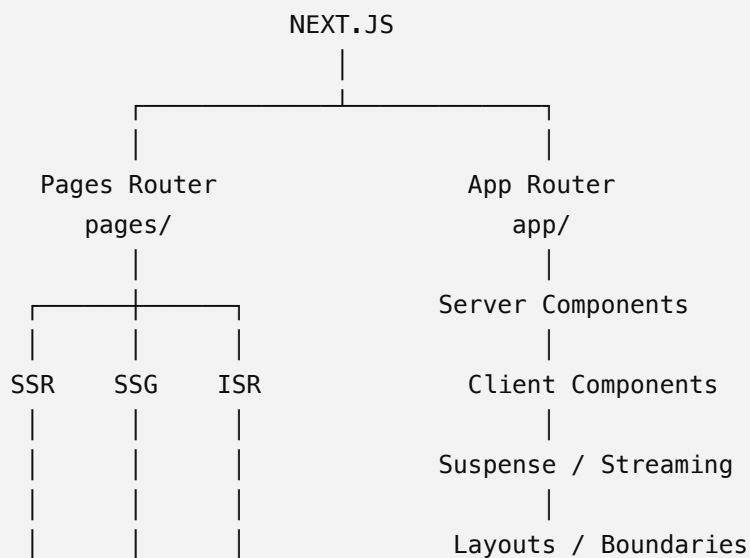
generateStaticParams()

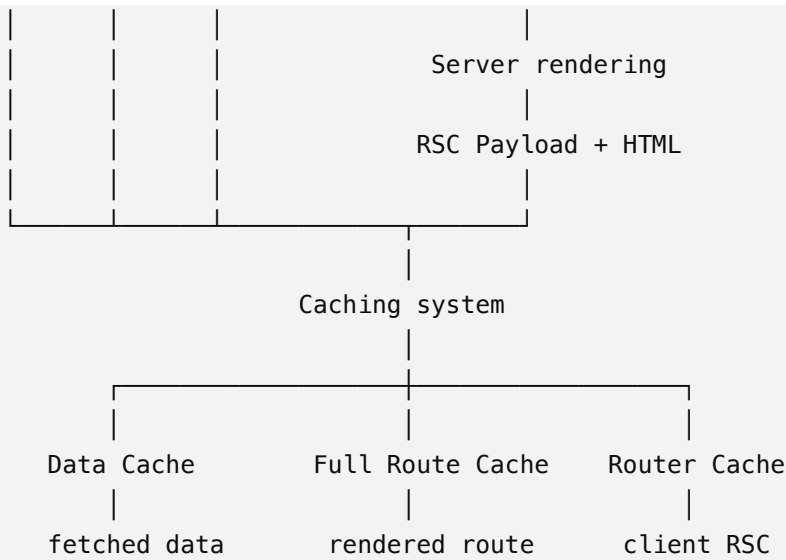


dynamic params to pre-render

31. The BIG rendering map

Now let's put everything together.





That's the mental model I want you to have.

32. One thing we must NOT mix up

This is probably the most important lesson of this entire section:

Rendering

answers:

How/when is the UI generated?

Data Cache

answers:

Can fetched data be reused?

Full Route Cache

answers:

Can the rendered route result be reused?

Router Cache

answers:

Can the browser's App Router reuse previously received RSC segments?

Revalidation

answers:

When should cached information be considered stale/refreshed?

Routing

answers:

Which route/component tree should handle this URL?

They're connected, but they're different questions.

33. Let's trace 3 real scenarios

This is where the whole thing clicks.

Scenario A — Static About page

/about



No dynamic data.

Conceptually:

```
Build
↓
Server Component
↓
render
↓
HTML + RSC Payload
↓
Full Route Cache
```



User request:

```
Browser
↓
cached route result
```



That's basically SSG/static rendering.

34. Scenario B — Dynamic dashboard

/dashboard



Suppose it depends on:

```
cookies
authentication
request-specific data
```



Conceptually:

```
Request
↓
Server
↓
dynamic rendering
↓
fetch/user-specific data
↓
render
↓
HTML + RSC Payload
↓
response
```



This is the modern equivalent of the **SSR mental model**.

35. Scenario C — Product page with revalidation

```
/products/123
```



Data can be cached for 60 seconds.

Conceptually:

```
Request
↓
Server Component
↓
fetch()
↓
Data Cache
↓
product data
↓
render route
↓
Full Route Cache
↓
HTML + RSC Payload
```



After revalidation:

```
stale
↓
regenerate
↓
new route output
```



That's ISR-like behavior.

36. And now client navigation

User goes:

```
/products
```



then:

```
/products/123
```



via:

```
<Link href="/products/123">
```

Conceptually:

```
Browser
|
| client navigation
↓
Router Cache
|
| HIT → reuse RSC segment
|
| MISS
↓
Server
↓
render/fetch
↓
RSC Payload
↓
Router Cache
```



↓
update React UI

Notice something:

The Router Cache is a client-navigation optimization.

It's not the same thing as the Full Route Cache.

37. So what happens during a hard refresh?

This is another excellent distinction.

User does:

Cmd + R



That's a new request.

The browser doesn't simply say:

"Oh, I have this route in my Router Cache, I'll use that."

Instead, a new request goes to the server.

Conceptually:

```
Browser
  ↓
GET /products
  ↓
Next.js
  ↓
Data/Full Route caching
  ↓
response
  ↓
browser
```



So:

```
Client navigation
  ↓
Router Cache matters heavily
```



while:

Hard refresh/direct request



↓
server-side rendering/caching path matters

This distinction is **very important for interviews**.

38. Where does hydration fit?

Another term you absolutely need.

Suppose the server sends HTML:

```
<button>Count: 0</button>
```

The browser can display it.

But the button needs JavaScript to respond to:

```
onClick
```



The client React code attaches behavior.

That's hydration.

Conceptually:

```
Server
↓
HTML
↓
Browser displays UI
↓
JS loads
↓
React hydrates Client Components
↓
interactive UI
```



But importantly:

Server Components themselves aren't hydrated like Client Components.

Client Components are where client-side React interactivity is needed.

39. Why "use client" matters

Consider:

```
export default function Product() {  
  return <ProductDetails />  
}
```

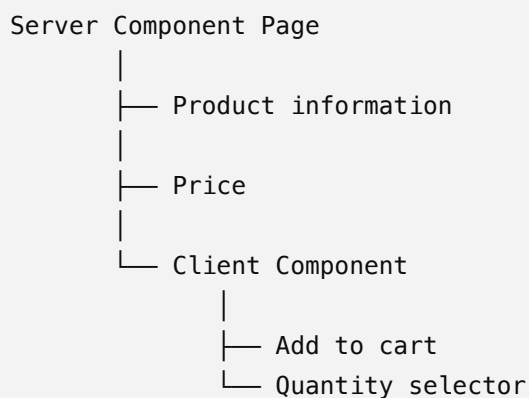
Server Component.

But:

```
"use client"  
  
export default function AddToCart() {  
  const [count, setCount] = useState(0)  
  
  return ...  
}
```

Client Component.

So a typical page might look like:



The server can render the overall tree while specific interactive islands use client-side React.

40. Where Server Actions fit

Now connect this to our previous lesson.

Suppose:

```
<form action={addProduct}>
```

and:

```
"use server"

async function addProduct(formData: FormData) {
  // mutation
}
```

The flow is roughly:

```
User submits form
  ↓
Server Action
  ↓
Database mutation
  ↓
cache invalidation/revalidation
  ↓
new server-side data
  ↓
RSC/UI update
  ↓
client UI reflects new state
```



That's why Server Actions fit naturally into App Router.

They're part of the same:

```
Server Components
+
RSC
+
caching
+
revalidation
+
client router
```



architecture.

41. Route Handlers vs API Routes

Another common confusion.

Pages Router

pages/api/products.ts



This is an:

API Route

Example:

/api/products



App Router

app/api/products/route.ts



This is a:

Route Handler

Example:

```
export async function GET() {  
  return Response.json(...)  
}
```

So:

Pages Router

↓

pages/api/*

↓

API Routes



versus:

App Router

↓

app/**/route.ts



↓
Route Handlers

42. Route Handler ≠ Server Component

Very important.

A route handler:

`app/api/products/route.ts`



is basically an HTTP endpoint.

It handles:

GET
POST
PUT
PATCH
DELETE



A Server Component:

`app/products/page.tsx`



is part of the UI rendering tree.

So:

Server Component
↓
UI

Route Handler
↓
HTTP response/API endpoint



They execute on the server, but their jobs are different.

43. Server Action vs Route Handler

Another interview favorite.

Route Handler

```
HTTP client
  ↓
POST /api/products
  ↓
Route Handler
```



Useful when you need an HTTP endpoint.

Server Action

```
React UI
  ↓
Server Action
  ↓
server function
```



Useful for UI-driven mutations.

So:

```
Route Handler = HTTP boundary

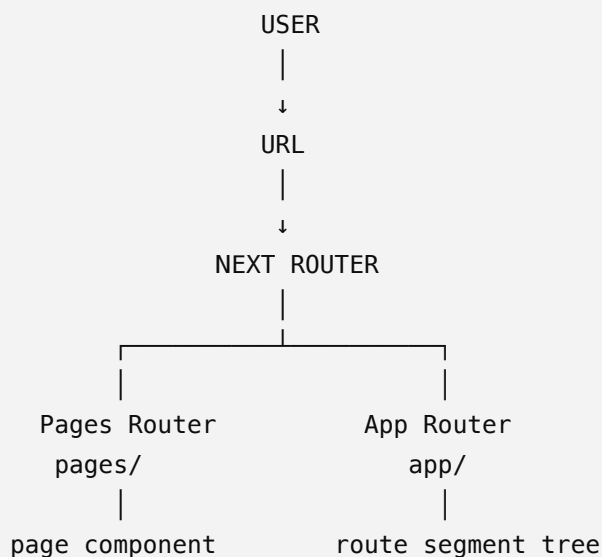
Server Action = React/server mutation boundary
```

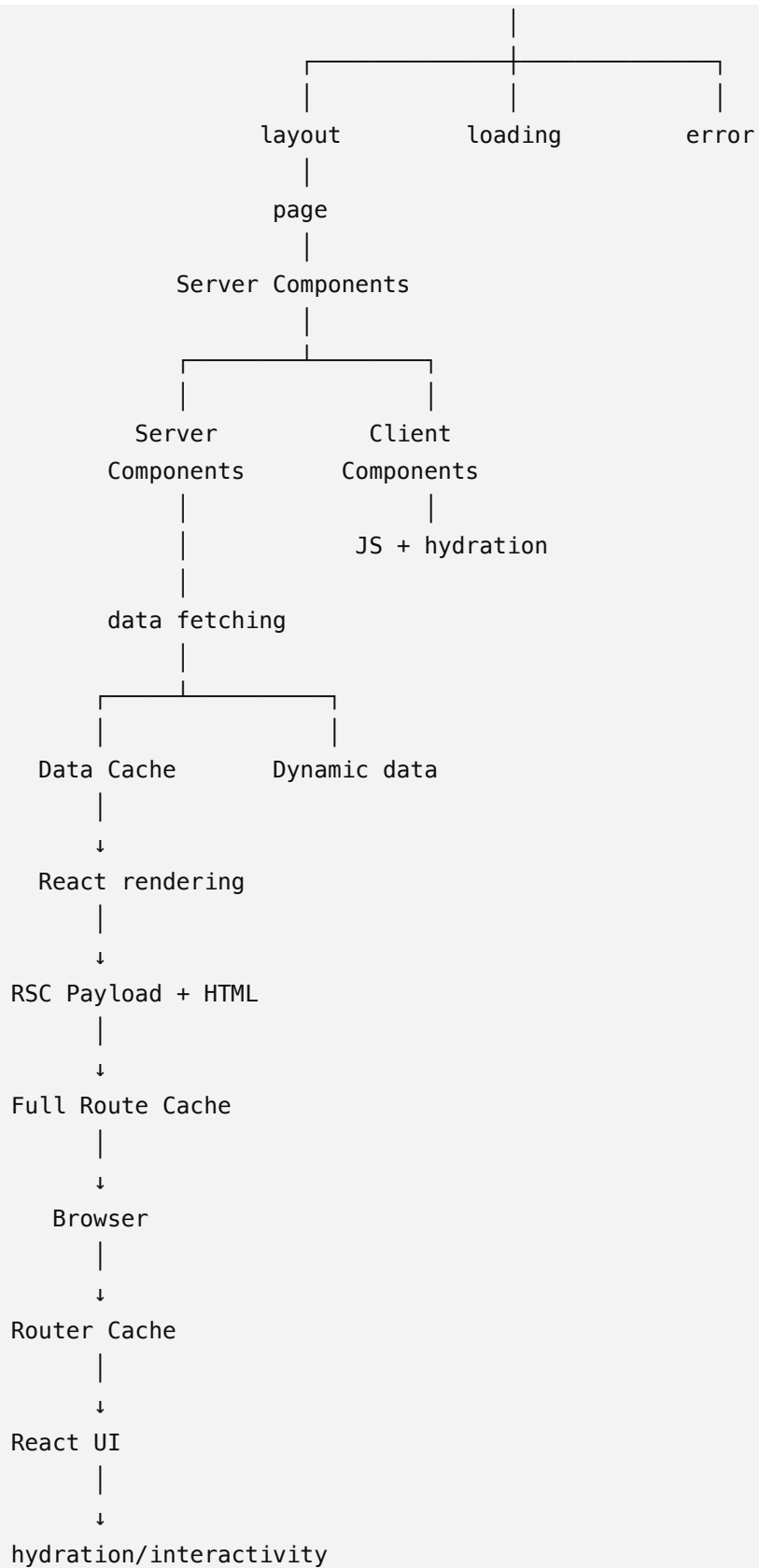


44. The complete architecture

Okay. Deep breath. 😊

Here's the **master picture**:





And mutations:

User interaction



Server Action / Route Handler



Database mutation



```

revalidation / invalidation
  ↓
Data Cache / Route Cache changes
  ↓
RSC result
  ↓
Router Cache/UI update

```

🔥 **THIS** is the architecture you're trying to understand.

45. One final correction to our earlier mental model

I want to tighten something from our previous cache lessons.

Don't memorize:

```

revalidatePath()
  ↓
"it invalidates Full Route Cache"

```



as if that were the entire story.

Instead think:

```

Mutation
  ↓
Which cached data became stale?
  ↓
Which routes depend on it?
  ↓
What server-side output needs to be regenerated/revalidated?
  ↓
What should the current client navigation/UI receive?

```



The exact behavior can differ based on **where the invalidation API is called and the current Next.js caching model**.

That's why we're separating:

```

Data Cache
Full Route Cache
Router Cache

```



before getting into rendering mechanics.

46. Our next section: Rendering — REALLY deep

Now that the architecture is connected, I would **not** jump straight to random definitions of SSR/SSG.

We should do the rendering section in this order:

5.10 — Next.js Rendering Deep Dive

Part 1 — Rendering fundamentals

- What "rendering" actually means
- React rendering vs browser rendering
- Server rendering vs client rendering
- HTML generation
- RSC rendering
- hydration
- hydration vs rendering

Part 2 — Static rendering

- SSG
- build-time rendering
- static routes
- `generateStaticParams`
- static data
- Full Route Cache
- what happens at build time

Part 3 — Dynamic rendering

- SSR mental model
- request-time rendering
- cookies
- headers
- search params
- authentication
- dynamic APIs
- what makes a route dynamic

Part 4 — ISR

- why ISR was created
- Pages Router ISR

- `revalidate`
- stale data
- regeneration
- App Router equivalent
- Data Cache + Full Route Cache interaction

Part 5 — Streaming

- Suspense
- `loading.tsx`
- progressive rendering
- RSC streaming
- HTML streaming
- why streaming isn't the same as SSR/SSG

Part 6 — Client rendering

- Client Components
- `"use client"`
- hydration
- browser rendering
- event handlers
- state
- when client rendering happens

Part 7 — Hybrid rendering

- Server Component → Client Component
- static shell + dynamic content
- server data + client interaction
- partial dynamic UI
- real-world product page example

Part 8 — Under-the-hood request lifecycle

We'll literally trace:

User enters URL

↓

DNS/CDN

↓

Next.js server

↓

Router

↓



```
route segment tree
  ↓
static or dynamic decision
  ↓
Data Cache
  ↓
Server Components
  ↓
React Server rendering
  ↓
RSC Payload
  ↓
HTML
  ↓
Full Route Cache
  ↓
network
  ↓
browser
  ↓
HTML display
  ↓
RSC reconciliation
  ↓
Client Component hydration
  ↓
Router Cache
  ↓
interactive app
```

And then we'll trace **the exact same page** for:

1. SSG
2. SSR
3. ISR
4. Client navigation
5. hard refresh
6. mutation + revalidation



That last comparison is where everything should finally **lock into place**. 🧠🔥



Initial navigation can involve:

```
HTML
+
RSC Payload
+
JS for Client Components....what do this mean ??
```



Yes — **you've spotted an important distinction.** 🙌 Let's slow this down because there are actually **three different things** involved:

```
HTML
RSC Payload
JavaScript for Client Components
```



And **Router Cache** stores the **RSC Payload**, not the **Client Component JavaScript**.

1. First: What is "JS for Client Components"?

Suppose your App Router page is:

```
</> TypeScript
// app/products/page.tsx

export default function ProductsPage() {
  return (
    <>
      <h1>Products</h1>
      <AddToCart />
    </>
  )
}
```



And:

```
</> TypeScript
// AddToCart.tsx

"use client"

export default function AddToCart() {
  const [count, setCount] = useState(0)

  return (
```



```
<button onClick={() => setCount(count + 1)}>
  Add {count}
</button>
)
}
```

The important thing is:

```
ProductsPage
  ↓
Server Component

AddToCart
  ↓
Client Component
```



The server can generate the **initial UI** for the Client Component.

But the browser needs the **JavaScript code** for `AddToCart` because the browser needs to execute:

```
</> JavaScript
onClick={() => ...}
```



and:

```
</> JavaScript
useState(...)
```



The server can't make the browser's button interactive just by sending HTML.

2. Think about the three pieces separately

When you initially load the page, conceptually you can think of the browser receiving:

① HTML

Something like:

```
HTML
<h1>Products</h1>

<button>
```



```
Add 0
</button>
```

This lets the browser **display the initial UI**.

② RSC Payload

The RSC Payload tells React about the result of the Server Component tree and how the server/client component boundaries fit together.

Very simplified:

```
ProductsPage
├── h1 → Products
└── AddToCart
    └── Client Component boundary
```

It's **not simply another copy of the HTML**.

It's information used by React/Next.js to reconstruct/update the component tree.

③ JavaScript for Client Components

This is the actual JavaScript code needed by the browser for Client Components.

For example, the bundled code containing the logic corresponding to:

```
</> TypeScript

"use client"

function AddToCart() {
  const [count, setCount] = useState(0)

  ...
}
```

The browser needs that JS so it can make:

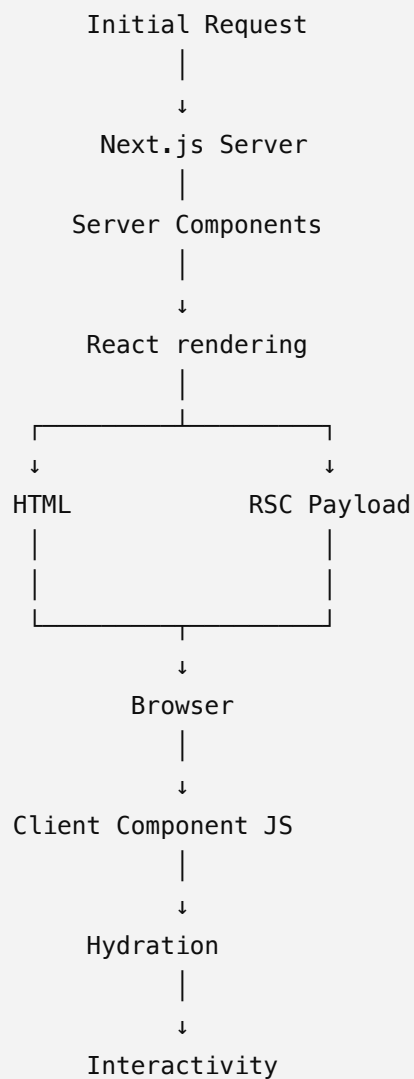
```
<button>
  Add 0
</button>
```

actually respond to:

```
CLICK
  ↓
setCount(1)
  ↓
React updates UI
```



3. So think of it like this



The **HTML** gets pixels/UI onto the screen.

The **Client Component JS** gives those components their client-side behavior.

The **RSC Payload** helps React understand the Server Component result/tree.

4. Now your Router Cache question 📌

You asked:

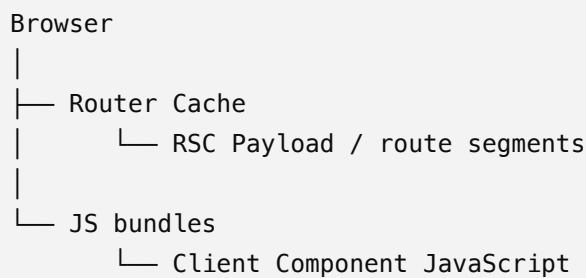
"Router Cache stores RSC Payload, right? So it doesn't load JS for Client Components in Router Cache?"

YES. Exactly. 🎯

The Router Cache is primarily a **client-side cache of RSC Payload / route segments**.

It does **not** act as a cache for the JavaScript bundles themselves.

Think:

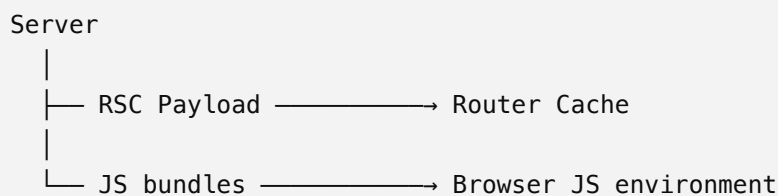


These are separate things.

5. So where does Client Component JS live?

The browser gets JavaScript bundles from the application's normal JS assets.

Conceptually:



The JS isn't stored **inside** the Router Cache.

The browser may cache JS assets separately using browser/platform caching mechanisms, but that's a different cache from Next.js's Router Cache.

6. Now let's see navigation

This is where your question becomes really interesting.

You're on:

```
/products
```



and click:

```
/products/123
```



using:

```
</> TypeScript
```



```
<Link href="/products/123">
```

Suppose Router Cache already contains the RSC segment needed for `/products/123` .

Then:

```
Click Link
```

```
↓
```

```
Client Router
```

```
↓
```

```
Router Cache
```

```
↓
```

```
RSC Payload found ✓
```

```
↓
```

```
React updates UI
```



There may be **no need to request that RSC payload again**.

But that's completely separate from Client Component JavaScript.

7. What if the page contains a Client Component?

Suppose `/products/123` contains:

```
ProductPage
```

```
|
```

```
├─
```

```
ProductDetails
```

```
← Server Component
```

```
|
```

```
└─
```

```
AddToCart
```

```
← Client Component
```



Router Cache might have:

```
Router Cache
```

```
└─ /products/123
```



└─ RSC Payload

But the browser still needs the JavaScript code for:

AddToCart



If that JS isn't already available in the browser, it needs to be loaded.

So:

Router Cache



|

└─ RSC Payload

✗ NOT Client Component JS

while:

Browser JS



|

└─ Client Component bundles

↓

React runtime

↓

interactivity

8. Here's the really important distinction

Router Cache answers:

"Do I already have the server-generated RSC result for this route segment?"

Client JS answers:

"Do I have the JavaScript code needed to execute this Client Component?"

They're solving different problems.

9. Example: navigating between two pages

Imagine:

/products

/products/123



/products/456

All pages use:

Navbar ← Client Component



and each page has:

AddToCart ← Client Component



After visiting `/products`, the browser may already have loaded the JS needed for those Client Components.

Then you navigate:

/products



/products/123



The Router Cache might provide:

RSC Payload for /products/123



And the browser may already have:

Client Component JS



So the navigation can be very efficient:

Click



Router Cache



RSC Payload



React



existing Client Component JS



UI update



No need to download the same JS again just because you navigated.

10. And this explains why Router Cache is useful

Imagine there was **no Router Cache**.

You navigate:

```
/products  
↓  
/products/123
```



Next.js would need to request the RSC result from the server.

Then:

```
Server  
↓  
RSC Payload  
↓  
Browser  
↓  
React
```



But with Router Cache:

```
Browser  
↓  
Router Cache  
↓  
RSC Payload already available  
↓  
React
```



Much faster.

11. But don't make this mistake ❌

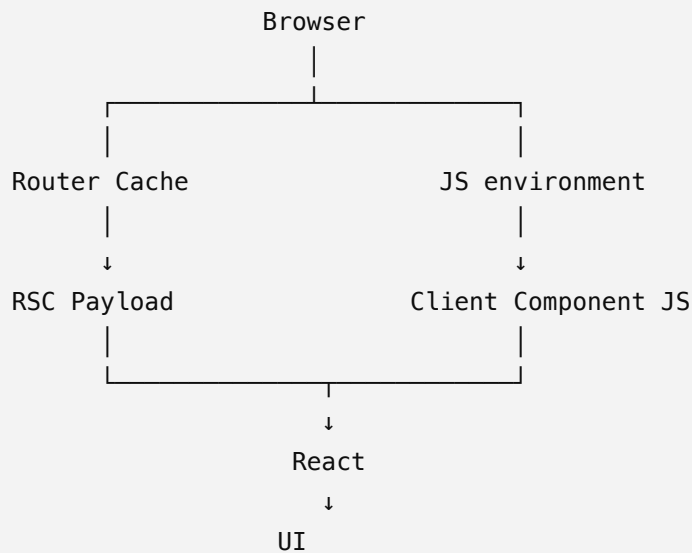
Don't think:

```
Router Cache  
├─ HTML  
├─ RSC Payload  
└─ Client JS
```



That's **not the mental model** you want.

Instead:

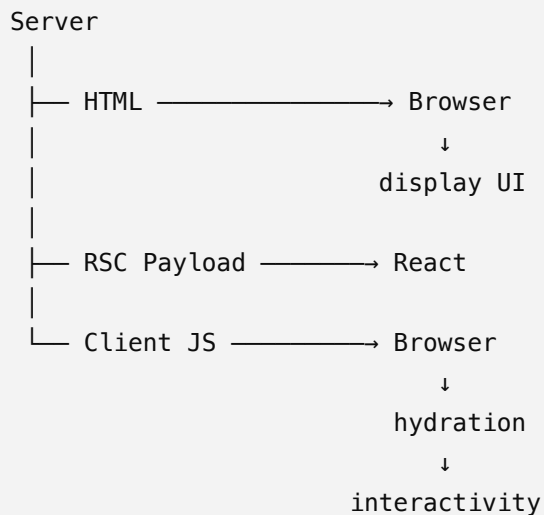


12. And where does HTML fit?

HTML is especially important for the **initial document load**.

Conceptually:

INITIAL LOAD



For subsequent App Router navigation, Next.js can request/receive **RSC payload rather than another complete HTML document**, which is one of the key benefits of the App Router architecture.

13. The easiest analogy 🧠

Think of a restaurant menu.

HTML

is like:

A printed menu you can immediately look at.

RSC Payload

is like:

Structured information telling the waiter/system what the menu structure and items are.

Client Component JS

is like:

The machinery needed to actually operate interactive things — e.g. a touchscreen ordering button.

Router Cache

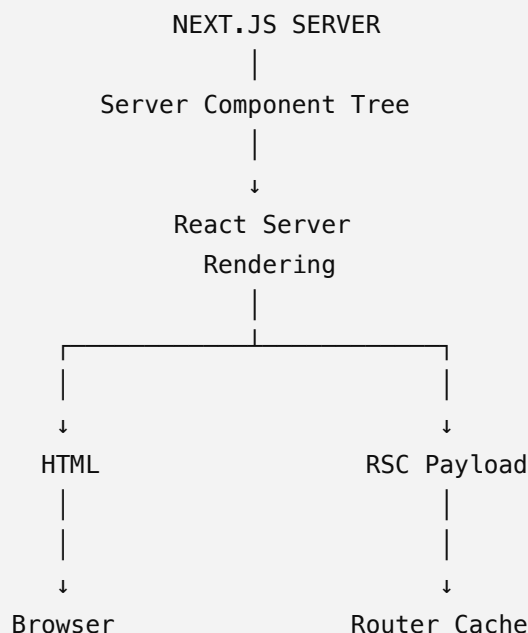
is like:

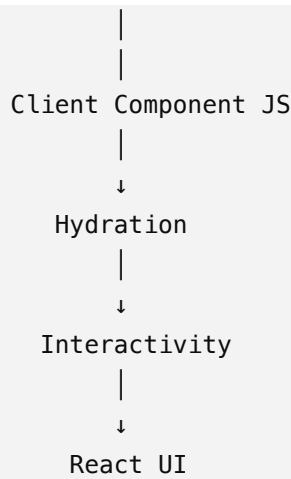
Keeping previously received structured menu sections nearby so you don't have to request them again.

The Router Cache doesn't magically store the touchscreen's operating software. 😊

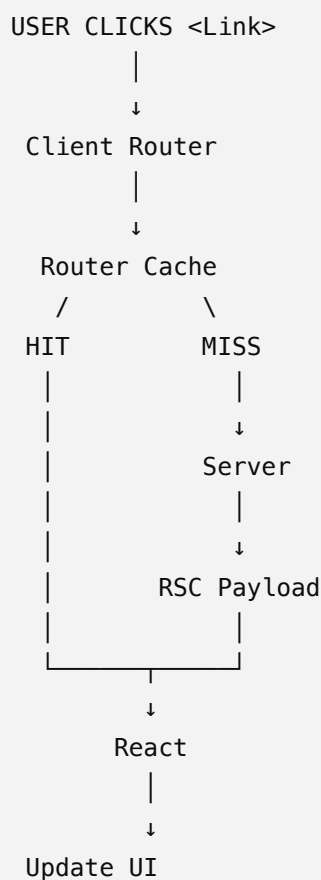
14. One final master diagram

This is the diagram I want you to remember:





And for navigation:



Meanwhile, **Client Component JS** is a **separate concern** and must be available in the browser for those components to be interactive.

The one-line memory trick:

HTML = initial visual UI, RSC Payload = Server Component result/structure, Client JS = interactivity, Router Cache = cached RSC segments for client navigation. 🎯

And this distinction is going to become **very important when we do "initial load vs client navigation vs hard refresh" under-the-hood**, because that's where HTML, RSC Payload, JS, Full Route Cache, and Router Cache all behave differently.





Exactly — this phrase is worth unpacking because “HTTP response” sounds abstract at first. 😊

🌐 What is an HTTP response?

When your browser asks a server for something, that's an **HTTP request**.

For example:



```
graph TD; Browser -- "GET /products" --> Server;
```

Browser

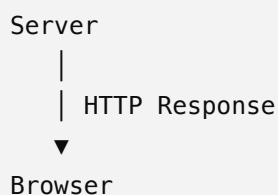
|

| GET /products

▼

Server

The server sends something back. That's the **HTTP response**.



```
graph TD; Server -- "HTTP Response" --> Browser;
```

Server

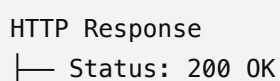
|

| HTTP Response

▼

Browser

A response can contain things like:



```
graph TD; HTTP_Response[HTTP Response] --- Status[Status: 200 OK];
```

HTTP Response

└─ Status: 200 OK

- └ Headers
- └ Body

For example:

```
</> http

HTTP/1.1 200 OK
Content-Type: text/html

<html>
  <body>
    <h1>Products</h1>
  </body>
</html>
```

So when I say:

CDN caches the HTTP response

I mean the CDN can save something like this:

```
GET /products
    ↓
Response:
200 OK
<html>Products...</html>
```

and reuse that response for another request.

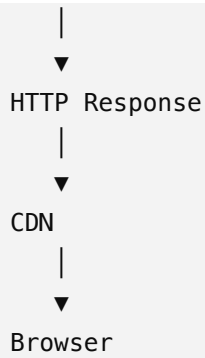
🔥 Without CDN cache

Suppose you visit:

```
https://example.com/products
```

The flow is:

```
Browser
  |
  | GET /products
  ▼
CDN
  |
  | cache MISS
  ▼
Next.js
```



Next.js might produce:

```
200 OK

<html>
  <h1>Products</h1>
  ...
</html>
```



The CDN can now say:

"This response is cacheable. I'll keep a copy."

So:

```
CDN Cache

/products
  ↓
200 OK
<html>Products...</html>
```



Next user requests `/products`

Now another user asks:

```
GET /products
```



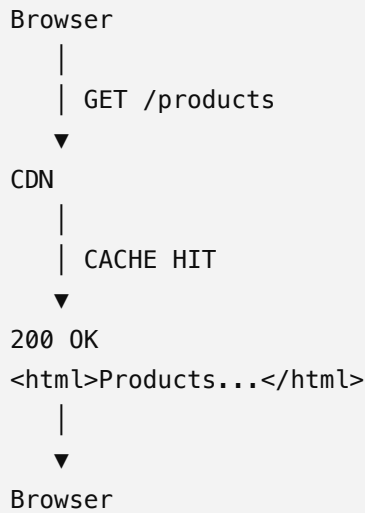
The CDN checks:

```
Do I already have a cached response for /products?
```



YES → Cache HIT 

So:



Next.js isn't contacted.

That's what we mean by:

The CDN serves the cached HTTP response.



Important distinction

The CDN isn't necessarily caching some special Next.js object like:

`FullRouteCacheObject`

Instead, from the CDN's perspective, it's dealing with the **HTTP request/response**.

Conceptually:

REQUEST

GET /products



RESPONSE

Status: 200

Headers: ...

Body: <html>...</html>



CDN saves the response



Later:

GET /products



CDN returns previously saved response



And this is different from Next.js Data Cache

This is where your previous questions connect.

Imagine:

Browser



CDN



Next.js



Data Cache



Database/API



Each cache operates at a different level.

Data Cache

Caches data:

/products-data



```
[
  { name: "iPhone" },
  { name: "MacBook" }
]
```



Full Route Cache

Caches rendered route output:

/products
↓
HTML + RSC Payload



CDN Cache

Caches the **HTTP response**:

GET /products
↓
HTTP Response
↓
200 + headers + response body



Router Cache

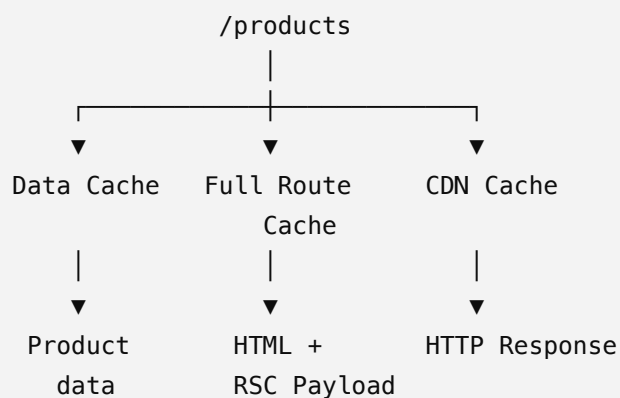
Caches **RSC route segments in the browser**:

/products
↓
RSC Payload / segments



The simplest mental model

Think about the same `/products` page:



And on the client:

Browser
└─ Router Cache
 ↓
 RSC data



So when you hear:

"CDN caches HTTP responses"

translate it in your head to:

"The CDN can save the server's entire response to a request and return that saved response directly to future users, without contacting the origin."

That's the key idea. 🗝️

And **this is why CDN cache and Next.js Full Route Cache are related but not the same thing** — both may ultimately involve the rendered page, but they sit at different layers and have different jobs.

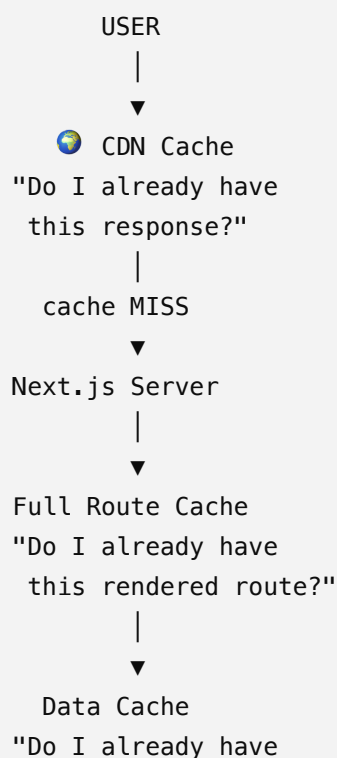


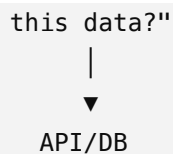
same job right caching..

Yes 😊 — at a very high level, they have the same goal: avoid doing work again.

But they don't have the **same job** because they cache **different things and sit in different places**.

Think of it like this:





The key difference

Cache	What is it avoiding?
Data Cache	Re-fetching data
Full Route Cache	Re-rendering a route
CDN Cache	Going all the way to your origin/server
Router Cache	Re-fetching RSC segments during client navigation

So imagine `/products` .

Without any cache



Lots of work. 😞

With Next.js caches



Next.js doesn't have to render again.

With CDN cache too

Browser



CDN Cache HIT 🚀



Browser



Next.js isn't even contacted.

🔥 So here's the distinction I want you to remember

Don't think:

"CDN cache and Next.js cache are competing caches."

Think:

"They're multiple layers of caching that prevent different pieces of work at different locations."

It's basically:

CDN



"Can I avoid contacting the server?"

Next.js Full Route Cache



"Can I avoid rendering the page?"

Next.js Data Cache



"Can I avoid fetching the data?"

Router Cache



"Can I avoid requesting RSC segments during navigation?"



So yes, same broad purpose — but different optimization points. 🎯

And this layered-cache idea is going to make **SSG vs SSR vs ISR** much easier to understand, because those determine *whether and where* a route can be cached.



ok

👍 Perfect.

Next logical piece is:

How Next.js decides: "This route is static or dynamic?"

That connects everything we've covered:

```
fetch()  
cookies()  
headers()  
searchParams  
no-store  
revalidate  
generateStaticParams()  
  ↓  
Static or Dynamic?  
  ↓  
SSG / SSR / ISR  
  ↓  
Full Route Cache  
  ↓  
CDN Cache  
  ↓  
Router Cache
```



That's the piece we should tackle next. 🧠🔥



ChatGPT can make mistakes. Check important info.